Projet 3: Requêter une base de données SQL

Méthodologie étape par étape

Comprendre les données

Identifier les types

De données et la

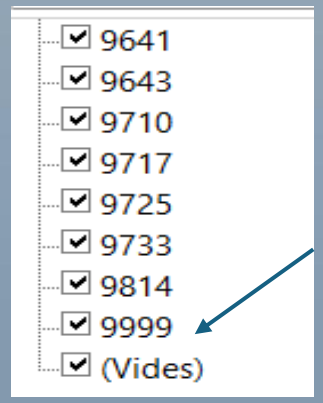
Nature et bien comprendre ce qu'ils représente (contrat, région département)

Code_dep_co	reg_code	reg_nom	aca_nom	dep_nom
1001	84	Auvergne-Rhô	Lyon	Ain
1002	84	Auvergne-Rhô	Lyon	Ain
1003	84	Auvergne-Rhô	Lyon	Ain
1004	84	Auvergne-Rhô	Lyon	Ain
1005	84	Auvergne-Rhô	Lyon	Ain
1006	84	Auvergne-Rhô	Lyon	Ain
1007	84	Auvergne-Rhô	Lyon	Ain
1008	84	Auvergne-Rhô	Lyon	Ain
1009	84	Auvergne-Rhô	Lyon	Ain

Filtré les colonnes

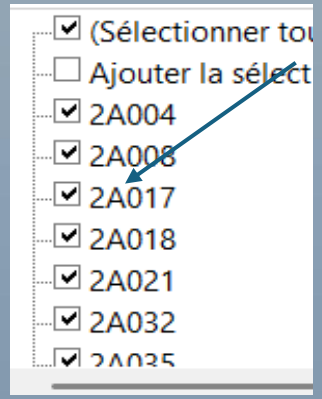
Pour détecter les

Pièges (valeurs vides). Bien adapter les contraintes (NULL qui autorise les valeurs vides)



Exemple de pièges

- Caractère textes cachés: bien adapter les types de données (CHAR, VARCHAR, INTEGER)



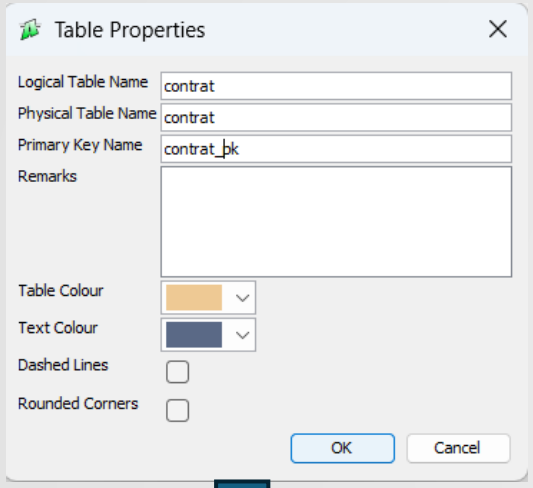
Dictionnaire des données

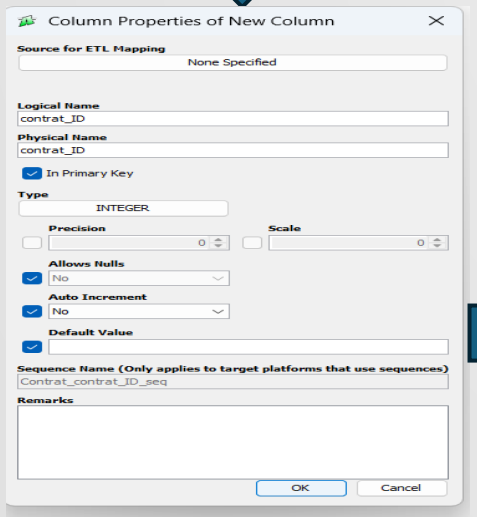
	Nom des colonnes	Type de données	Taille	Clé	Description
CONTRAT.CSV	Contrat_ID	INT		Clé primaire	Id unique pour les contrats
	No_voie	VARCHAR	50		Numéro dans la voie pour l'adresse du logement assuré
	B_T_Q	CHAR	1		Indicateur éventuel de répétition pour l'adresse du logement assuré sur un caractère
	Type_de_voie	VARCHAR	20		Type de voie pour l'adresse du logement assuré: rue, av (Avenue), rte (Route), ...
	Voie	VARCHAR	100		Libellé de la voie pour l'adresse du logement assuré
	Code_dep_code_commune	CHAR	6	Clé secondaire	Concaténation du code département et code commune pour avoir une clé unique
	Code_postal	INT			Code postal pour l'adresse du logement assuré
	Surface	INT			surface du bien
	Type_local	VARCHAR	20		typologie du logement (appartement ou maison)
	Occupation	VARCHAR	20		qualité de l'occupant (propriétaire ou locataire)
	Type_contrat	VARCHAR	50		type de contrat (residence principale, secondaire, location)
	Formule	VARCHAR	20		type de formule (classique ou integrale)
	Valeur_declaréé_biens	VARCHAR	20		Tranche de la valeur du bien
	Prix_cotisation_mensuel	INT			prix cotisation mensuel
REGION.CSV	Code_dep_code_commune	VARCHAR	10	Clé primaire	Concaténation du code département et code commune pour avoir une clé unique
	reg_code	INT			code de la région
	reg_nom	VARCHAR	50		nom de la région
	aca_nom	VARCHAR	50		nom de l'academie
	dep_nom	VARCHAR	50		nom du département
	com_nom_maj_court	VARCHAR	50		nom de la commune
	dep_code	CHAR	3		numéro du département
	dep_nom_num	VARCHAR	50		concatenation du nom du département et du numéro du département

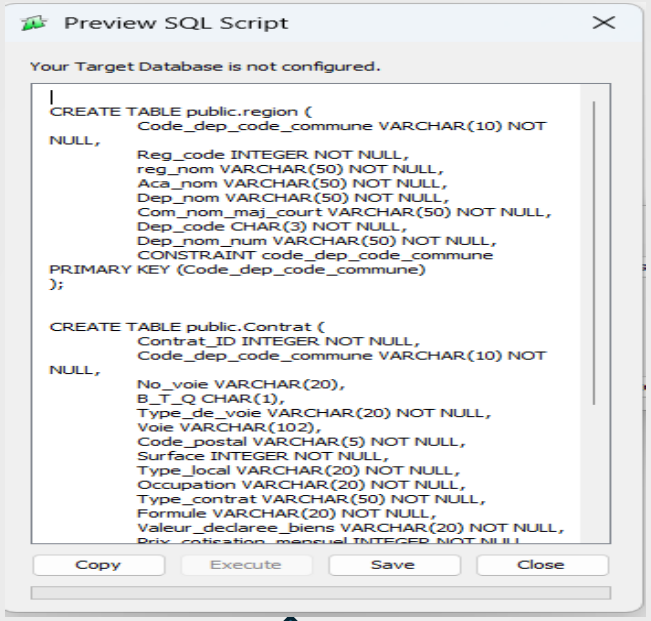
Remplir le dictionnaire des données

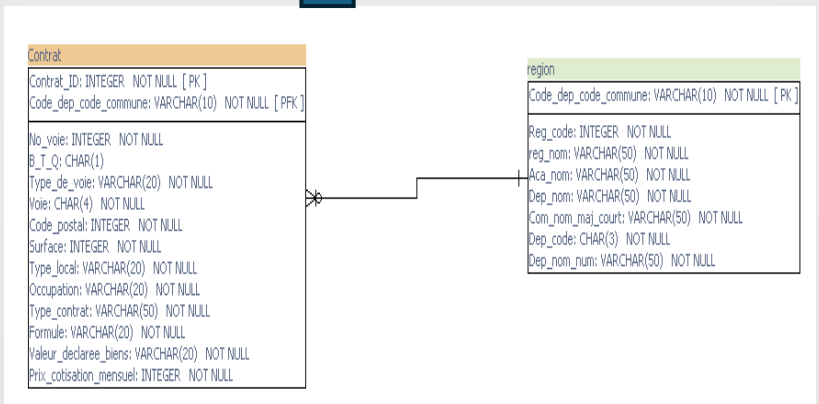
- **Nom des colonnes**
- **Type de données** (INT, CHAR, VARCHAR)
- **Définir la taille des type de données**
- **Déterminer la clé** : si elle est primaire ou étrangère
- **Donnée une description**: en relation avec la colonne est une étape importante pour la compréhension métier des champs entre équipe.

Réaliser une Architecture





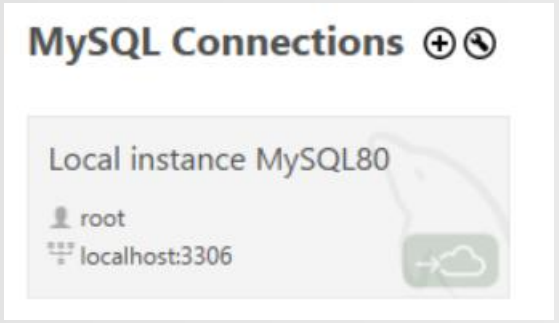




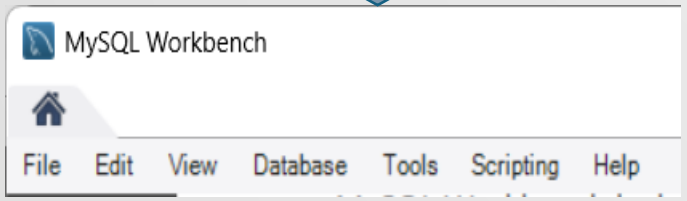
Retranscrire sur SQL architecture les données du dictionnaire:

- Crée une table
- Création des colonnes (adapter le type de données)
- Faire la jointure : entre la clé primaire et la clé étrangère
- Générer le code SQL

Télécharger une SGBD et importer des données



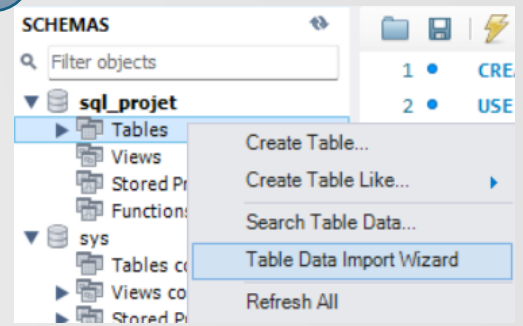
Télécharger
MYSQL
WORKBCHENCH



1

```
1 •  CREATE DATABASE SQL_projet;
2 •  USE SQL_projet;
```

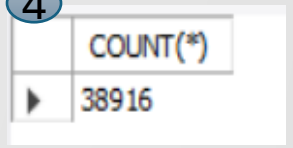
2



3



4



1) **Crée la base de données :** Sql_projet avec le code SQL
2) **Une fois la base crée:** importer les tables
3) **Charger le fichier** CSV et préciser l'encodage UTF8
4) **Vérifier l'import** :avec le code SQL « SELECT COUNT(*) FROM région

Importer des données

- Il existe une autre manière d'importer des données

1) Création d'une table

```
CREATE TABLE contrat (
    Contrat_ID INT PRIMARY KEY,
    No_voie VARCHAR(10) NULL,
    B_T_Q CHAR(1),
    Type_de_voie VARCHAR(20) NOT NULL,
    Voie VARCHAR(100) NULL,
    Code_dep_code_commune CHAR(6) NOT NULL,
    Code_postal VARCHAR(5) NOT NULL,
    Surface INT NOT NULL,
    Type_local VARCHAR(20) NOT NULL,
    Occupation VARCHAR(20) NOT NULL,
    Type_contrat VARCHAR(50) NOT NULL,
    Formule CHAR(9) NOT NULL,
    Valeur_declaréé_biens VARCHAR(20) NOT NULL,
    Prix_cotisation_mensuel INT NOT NULL
);
```

2) importer les données

```
LOAD DATA INFILE "C:\Users\DELL\Desktop\Projet 3\Contrat.csv"
INTO TABLE contrat
CHARACTER SET utf8mb4
FIELDS TERMINATED BY ';'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

Les requêtes simple

```
SELECT contrat_ID, Surface
FROM contrat
WHERE code_postal =
92100 ;
```

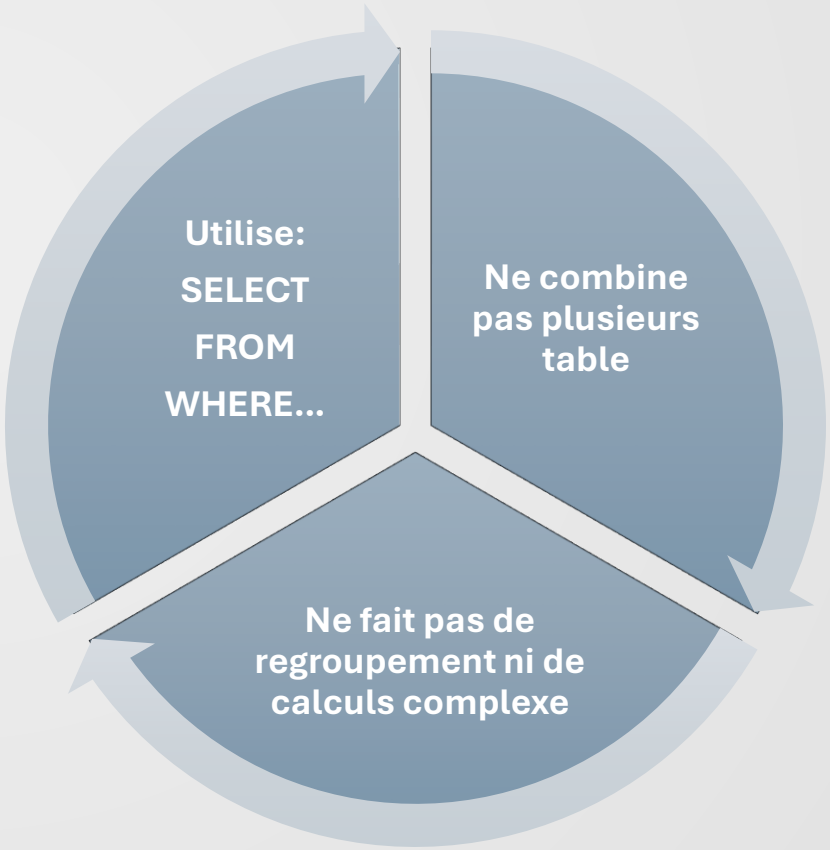
Contrat_ID	Surface
123026	61
123027	31
123028	17
123030	78
123030	109
123033	87
123034	80

```
SELECT DISTINCT reg_nom
FROM region ;
```

reg_nom
Auvergne-RhÃ´ne-Alpes
Hauts-de-France
Provence-Alpes-CÃ´te d'Azur
Grand Est
Occitanie
Normandie
Nouvelle-Aquitaine

```
SELECT COUNT (contrat_ID) AS
nb_contrats_residences_principales
FROM contrat
Where Type_local = 'Résidence
principale' ;
```

nb_contrats_residences_principales
25620



Requêtes avec agrégation

```
SELECT Contrat_ID, Surface
FROM contrat1
ORDER BY Surface DESC
LIMIT 5 ;
```

Contrat_ID	Surface
104211	815
105463	742
130878	595
100822	570
109872	559

```
SELECT
AVG(Prix_cotisation_mensuel) AS
Prix_moyen_cotisation
FROM contrat ;
```

Prix_moyen_cotisation
19.3287

```
SELECT valeur_declaree_biens
COUNT(Contrat_ID) AS
nb_contrat
FROM Contrat
GROUP BY
Valeur_declaree_bien
ORDER BY nb_contrats DESC ;
```

Valeur_declaréé_biens	nb_contrats
0-25000	22720
25000-50000	6815
50000-100000	696
100000+	104

Les requêtes avec agrégation:
Fonctionne de la même manière que les requêtes simples, la différence est que les calculs sont un peu plus poussé.



SUM(): calcul de la somme
COUNT(): compte le nombre de valeur
Min(): minimum/maximum
ORDER BY(): trie par défaut ou ordre croissant
DESC(): ordre décroissant
GROUP BY(): regrouper les lignes d'une table selon une ou plusieurs colonnes

Requêtes avec jointure

- **Les requêtes avec jointure:** servent à relier plusieurs tables entre elles, grâce à une colonne commune (souvent une clé primaire et une clé étrangère)
- **Les données:** sont réparties dans plusieurs tables
- **La jointure permet:** de croiser les informations pour obtenir une vue complète
- Type de jointure (LEFT JOIN())

```
SELECT r.dep_code,
AVG(c.Prix_cotisation_mensuel)
AS prix_moyen_cotisation
FROM contrat c
JOIN region r
ON c.Code_dep_code_commune
= r.Code_dep_code_commune
GROUP BY r.dep_code
ORDER BY prix_moyen_cotisation
DESC
LIMIT 10 ;
```

dep_code	prix_moyen_cotisation
75	36.3991
92	26.2694
94	19.8284
78	18.8858
69	18.4912
1	18.2389
6	18.1422
17	17.3173
74	17.1210
2A	17.0991

```
SELECT r.com_nom_maj_court
as commune,
COUNT(c.contrat_ID) as
nb_contrats
FROM contrat c
JOIN region r
ON c.code_dep_code_commune
= r. code_dep_code_commune
GROUP BY
r.com_nom_maj_court
HAVING
COUNT(c.contrat_ID)>=150
ORDER BY nb_contrats DESC ;
```

commune	nb_contrats
PARIS 18	515
PARIS 17	468
PARIS 15	407
PARIS 16	394
NICE	386
PARIS 11	381
BORDEAUX	302
PARIS 20	302
NANTES	291
PARIS 19	266

```
SELECT r.reg_nom as region,
COUNT(c.contrat_ID) as
nb_contrats
FROM contrat c
JOIN region r

ON
c.code_dep_code_commune =
r.code_dep_code_commune
GROUP BY r.reg_nom
ORDER BY nb_contrats DESC ;
```

region	nb_contrats
Ile-de-France	14177
Provence-Alpes-CÃ´te d'Azur	3279
Auvergne-RhÃ´ne-Alpes	3042
Nouvelle-Aquitaine	2038
Occitanie	1609
Pays de la Loire	1196
Hauts-de-France	1189
Bretagne	947
Normandie	824
Grand Est	769
Centre-Val de Loire	598
Bourgogne-Franche-Comté	293

```
SELECT AVG(Surface) AS
surface_moyenne_paris
FROM contrat c
JOIN region r
ON c.Code_dep_code_commune
= r.Code_dep_code_commune
WHERE r.aca_nom = 'PARIS';
```

surface_moyenne_paris
51.7695

```
SELECT c. Contrat_ID,
c.Type_contrat,   c.Formule
FROM contrat c
JOIN region r
 ON
c.Code_dep_code_commune =
r.Code_dep_code_commune
WHERE c.Type_local = 'Maison'
AND r.dep_code = '71';
```

Contrat_ID	Type_contrat	Formule
114768	Residence principale	Integral
114779	Residence principale	Classique
114782	Residence principale	Classique
114812	Residence principale	Integral

```
SELECT COUNT (contrat_ID) AS
nb_contrats_integral_PaysLoire
 FROM contrat c
LEFT JOIN region r
ON c.Code_dep_code_commune
= r.Code_dep_code_commune
WHERE Formule = 'intégral'
AND reg_nom = 'Pays de la Loire';
```

nb_contrats_integral_PaysLoire
589